

# Dynamic Programming & Reinforcement Learning: Assignment 3

Vrije Universiteit Amsterdam - Group 14 - 08/12/2023

Danila Romanov  
2792789

Ángel A. Gil  
2788988

## 1 Introduction

This report presents the design and implementation of a Monte Carlo Tree Search (MCTS) algorithm tailored to outperform a random agent in the game of Connect 4.

Firstly, we describe the problem, detailing the initial state and the structure of the Connect4 class. This is followed by an in-depth examination of our MCTS algorithm, elaborating on its exploration and update mechanisms. Subsequently, we provide a detailed analysis of a game, comparing theoretical winning probabilities with those estimated by our MCTS. The report concludes with statistical evaluations, offering insights into the convergence process of our approach.

*Disclaimer:* The code submitted alongside this report was developed independently with consultative support from ChatGPT-4V. While the LLM did not significantly contribute to resolving complex aspects of the problem, it aided in refining the overall structure and expediting the styling of our implementation.

## 2 Problem Description and encoding

Connect Four is a two-player board game, where the players choose a color and then take turns dropping colored discs into a seven-column, six-row vertically suspended grid. The pieces fall straight down, occupying the lowest available space within the column. The objective of the game is to be the first to form a horizontal, vertical, or diagonal line of four of one's own discs.

In the assignment, we are given a game already in progress, as depicted in Figure 1. This initial state limits available actions to playing in either column 1 or 7. Our agent, represented by the dark discs, is next to move. The game board is encoded as a two-dimensional numpy array, where rows are the first dimension. The encoding uses 0 for empty cells, 1 for the MCTS agent's discs, and 2 for the discs of the random agent.

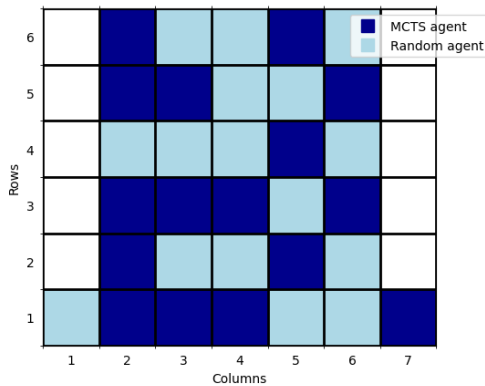


Figure 1: Initial state of the game. MCTS agent to play

Additionally, a Connect4Visualizer class was developed, capable of generating images like Figure 1. This class features a dynamically adaptive title indicating the next player's turn and automatically recognizes a player's victory, highlighting the winning sequence in yellow.

## 3 Our MCTS implementation

The implemented Monte Carlo Tree Search (MCTS) algorithm employs the Upper Confidence Bound for Trees (UCT) selection strategy. This approach integrates the Upper Confidence Bounds (UCB) for the selection phase and uses a random rollout from leaf nodes until a terminal game state is reached. Our approach makes use of 2 classes: *Node* and *MCTS*.

**Node Representation:** Each Node object in our MCTS represents a state of the Connect 4 game. It contains information about the game state at that node, the parent node (if any), the move that led to this node, the children nodes (initially empty), the total reward accumulated at this node, the number of visits to this node, a list of untried moves from this state, and the player's turn.

**Monte Carlo Tree Search with UCT:**

- (1) **Selection (UCT):** Beginning at the root, the tree is traversed to the leaf by selecting nodes that maximize the UCT value, striking a balance between exploration and exploitation. The UCT value is derived as  $\frac{\text{total reward}}{\text{visits}} + \sqrt{\frac{2 \times \ln(\text{total parent visits})}{\text{visits}}}$ , where a higher value indicates a more promising node to explore.
- (2) **Expansion:** Upon reaching a leaf node, the algorithm expands the tree by adding a new child node for all untried moves, creating a new possible game state from the current game configuration.
- (3) **Simulation:** A simulation or 'payout' commences from the newly added node, randomly selecting moves until the game concludes. The game's outcome (1 for a win, -1 for a loss, and 0 for a draw) is noted as a numerical value for the backpropagation phase.
- (4) **Backpropagation:** The simulation outcome propagates back up the tree, updating each node's total reward and visit count. The total reward is the sum of all previous game outcomes and is then used to update the tree nodes along the path taken by the simulation.

**Action Determination:** After completing the specified number of iterations, the algorithm selects the move corresponding to the child of the root node with the highest average reward. This is the move that the MCTS algorithm determines to be the most promising based on the simulations.

In short, the Node class serves as the fundamental unit that represents each state within the search tree and contains the selection, expansion, simulation and backpropagation methods. The MCTS class performs the iterations, repeatedly building out the tree and

refining the value estimates for each node. It does this by making use of *Node* objects. Once done iterating, it chooses the move associated with the child node of the root with the highest average reward (total reward divided by visits).

#### 4 Average expected reward: Theory vs MCTS

Figure 2 showcases a game being played out along with the root values for a 1000 iterations MCTS agent. For convenience, we picked the opponent moves in this demonstration. Let's assess whether the root node values make sense by comparing these with the actual probabilities of winning.

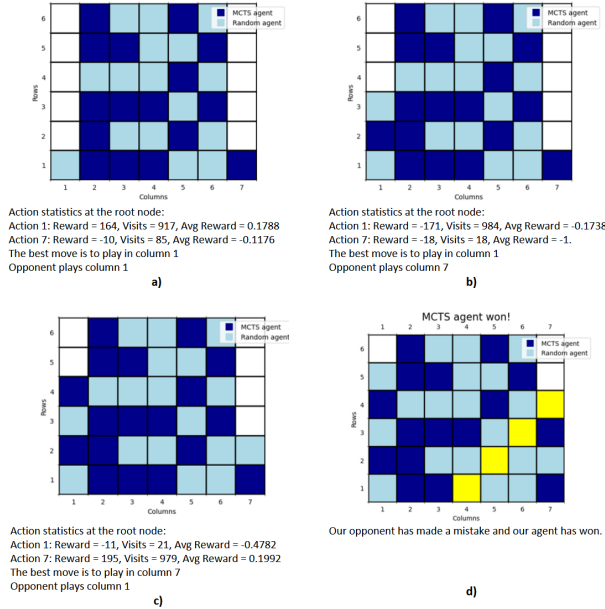


Figure 2: Game visualization with root values

Hence, we are going to discuss the possible scenarios making use of the sub-cases in Figure 2. The notation for characterizing any move will be of the form  $(inm)$  where  $i \in \{A, O\}$ ,  $n \in \mathbb{Z}^+$ ,  $m \in \{1, 7\}$ , representing the Agent or Opponent move, turn number (starting from 1 for the initial position) and column played:

- **(a) First turn** The preferred option is column 1, (A11). This makes sense since choosing column 7, (A17), would enable an instant win by the random agent by (O27). Since the latter would play (O21) half of the times, this also enables an instant win for our agent with (A31) which, again, given that we implemented random rollout, won't be always chosen. We could get into more depth, but for the sake of brevity let's just say that (A11) is the safe and best option as spotted by the agent. If the opponent plays (O27) now, (A31) would be

an instant win that our agent wouldn't miss so let's assume opponent plays (O21) next.

- **(b) Third turn** In this move we are forced to play (A31), not only to block the opponent, but also because (A37) would make us automatically lose in the next turn, as well captured by the root node value. (A31) still leaves us slightly negative in the random play-out because the remaining winning combinations of the opponent (at height 3 and 5 in column 7) come earlier than ours (height 4 and 6). Let's assume (O47). (O41) leads to a forced draw sequence - if the opponent were to play optimal - after (A51) to avoid enabling an instant win in column 7. Our agent was also able to spot the ideal moves in the latter scenario.
- **(c) Fifth turn** Easy to spot by our agent that we need to block (A57) as half of the times we are instantly losing if we do not do so (half of the times because random opponent, otherwise certain lost). Let's assume opponent also blocks us (O67) to keep stretching the example - O61 is an automatic win for us.
- **(d) Seventh turn** Our opponent has made a mistake and the MCTS agent capitalizes on that by selecting action 7 as the best move.

#### 5 Insights on the algorithm's convergence

In Figure 3, we can see that as the number of iterations goes up, the expected reward slightly goes up and the standard deviation goes down. This happens as more iterations allows for more of the correct paths to be found and for the model to more often give the accurate decisions. We found that at 1000 iterations there is sufficient convergence and the standard deviation is 0.1.

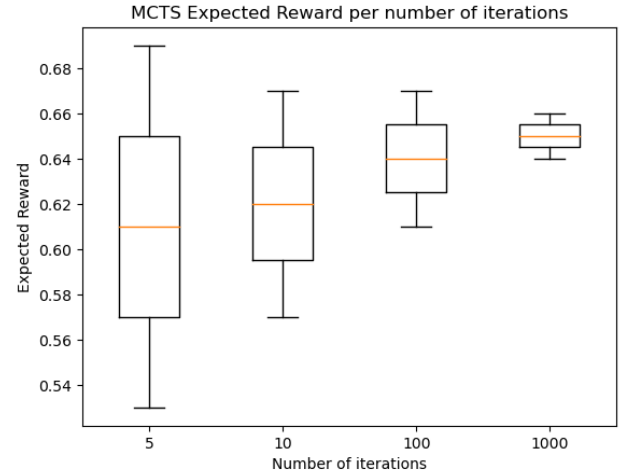


Figure 3: Convergence with number of iterations